```
// ============================================================================
// IoT Based Valve Control - Single-file (PIC18F25K22, XC8 v2.x)
// Modes: Auto (PowerDetect=HIGH), Manual (PowerDetect=LOW)
// Bluetooth (HC-05 @ 9600): PULSE##, STS, RESETMEM
// OLED SSD1306 128x64 I2C 0x3C (minimal 6x8 uppercase font)
// Limit switch on INT0 with debounce; relays via BC547; EEPROM-backed target N
// Manual mode per latest spec: no default close, no EEPROM writes, direction
// chosen by OpenFeedback/CloseFeedback.
// ============================================================================

#include <xc.h>
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include <stdarg.h>
#include <stdio.h>
#include <ctype.h>

// ========================= CONFIG BITS ======================================
#pragma config FOSC = INTIO67, PLLCFG = OFF, PRICLKEN = ON, FCMEN = OFF, IESO = OFF
#pragma config PWRTEN = OFF, BOREN = SBORDIS, BORV = 285
#pragma config WDTEN = OFF, WDTPS = 1024
#pragma config CCP2MX = PORTC1, PBADEN = OFF, CCP3MX = PORTB5, HFOFST = ON
#pragma config T3CMX = PORTC0, P2BMX = PORTB5, MCLRE = EXTMCLR
#pragma config STVREN = ON, LVP = OFF, XINST = OFF
// ============================================================================

#define _XTAL_FREQ 16000000UL
#define MS_TICK_HZ 1000U
```

```
// ======================== Pin Map ========================================
// Inputs
#define PIN_POWER_DETECT   PORTAbits.RA0
#define TRIS_POWER_DETECT  TRISAbits.TRISA0

#define PIN_TRIGGER_AUTO   PORTAbits.RA1
#define TRIS_TRIGGER_AUTO  TRISAbits.TRISA1

#define PIN_MANUAL_TRIG    PORTAbits.RA2
#define TRIS_MANUAL_TRIG   TRISAbits.TRISA2

#define PIN_OPEN_FB        PORTAbits.RA3   // "Open Feedback" input
#define TRIS_OPEN_FB       TRISAbits.TRISA3

#define PIN_CLOSE_FB       PORTAbits.RA4   // "Close Feedback" input
#define TRIS_CLOSE_FB      TRISAbits.TRISA4

#define PIN_LIMIT_INT0     PORTBbits.RB0   // Limit switch pulses on INT0
#define TRIS_LIMIT_INT0    TRISBbits.TRISB0

// Outputs
#define LAT_OPEN_RELAY     LATBbits.LATB1
#define TRIS_OPEN_RELAY    TRISBbits.TRISB1

#define LAT_CLOSE_RELAY    LATBbits.LATB2
#define TRIS_CLOSE_RELAY   TRISBbits.TRISB2

#define LAT_OPEN_STATUS    LATBbits.LATB3  // status out to other device
```

```
#define TRIS_OPEN_STATUS   TRISBbits.TRISB3

#define LAT_CLOSE_STATUS   LATBbits.LATB4  // status out to other device
#define TRIS_CLOSE_STATUS  TRISBbits.TRISB4

#define LAT_LED_HEART      LATBbits.LATB5
#define TRIS_LED_HEART     TRISBbits.TRISB5
// ============================================================================

// ========================= I2C (MSSP1) ====================================
static void i2c_init(uint32_t i2c_clk_hz){
   TRISCbits.TRISC3 = 1; // SCL
   TRISCbits.TRISC4 = 1; // SDA
   SSP1STAT = 0x00;
   SSP1CON1 = 0x28; // I2C Master
   SSP1CON2 = 0x00;
   uint32_t add = (_XTAL_FREQ/(4UL*i2c_clk_hz)) - 1UL;
   SSP1ADD = (uint8_t)add;
}
static inline void i2c_wait_idle(void){ while ((SSP1CON2 & 0x1F) || (SSP1STATbits.R_nW)); }
static bool i2c_start(uint8_t addr_rw){
   i2c_wait_idle(); SSP1CON2bits.SEN=1; while(SSP1CON2bits.SEN);
   i2c_wait_idle(); SSP1BUF=addr_rw; while(SSP1STATbits.BF); while(SSP1STATbits.R_nW);
   return !SSP1CON2bits.ACKSTAT;
}
static bool i2c_write(uint8_t v){
   i2c_wait_idle(); SSP1BUF=v; while(SSP1STATbits.BF); while(SSP1STATbits.R_nW);
   return !SSP1CON2bits.ACKSTAT;
}
```

```
static void i2c_stop(void){ i2c_wait_idle(); SSP1CON2bits.PEN=1; while(SSP1CON2bits.PEN); }

// ========================= UART (EUSART1) ===================================
static void uart_init(uint32_t baud){
   TRISCbits.TRISC6=1; TRISCbits.TRISC7=1;
   TXSTA1=0x24; RCSTA1=0x90; BAUDCON1=0x08; // BRGH, TXEN, SPEN, CREN, BRG16
   uint16_t spbrg=(uint16_t)((_XTAL_FREQ/(4UL*baud))-1UL);
   SPBRG1=(uint8_t)spbrg; SPBRGH1=(uint8_t)(spbrg>>8);
   TRISCbits.TRISC6=0;
}
static void uart_putc(char c){ while(!PIR1bits.TX1IF); TXREG1=c; }
static void uart_puts(const char*s){ while(*s){ if(*s=='\n') uart_putc('\r'); uart_putc(*s++);} }
static bool uart_kbhit(void){ return PIR1bits.RC1IF!=0; }
static char uart_getc(void){ if(RCSTA1bits.OERR){RCSTA1bits.CREN=0;RCSTA1bits.CREN=1;}
while(!uart_kbhit()); return RCREG1; }

// ========================= EEPROM utils ======================================
#define EEPROM_ADDR_MAGIC 0x00
#define EEPROM_ADDR_COUNT 0x02
#define EEPROM_ADDR_CSUM  0x04
#define EEPROM_MAGIC_VAL  0x5Au
static void eew_wait(void){ while(EECON1bits.WR); }
static void eew_write(uint8_t a, uint8_t v){
   EEADR=a; EEDATA=v; EECON1bits.EEPGD=0; EECON1bits.CFGS=0;
   INTCONbits.GIE=0; EECON1bits.WREN=1; EECON2=0x55; EECON2=0xAA; EECON1bits.WR=1;
EECON1bits.WREN=0; eew_wait(); INTCONbits.GIE=1;
}
static uint8_t eer_read(uint8_t a){ EEADR=a; EECON1bits.EEPGD=0; EECON1bits.CFGS=0;
EECON1bits.RD=1; return EEDATA; }
static uint8_t csum(uint8_t a,uint8_t b,uint8_t c){ return (uint8_t)((uint16_t)a+b+c+0x3D); }
```

```
static void eeprom_save_target(uint16_t n){
  uint8_t lo=(uint8_t)n, hi=(uint8_t)(n>>8);
  eew_write(EEPROM_ADDR_MAGIC, EEPROM_MAGIC_VAL);
  eew_write(EEPROM_ADDR_COUNT, lo);
  eew_write(EEPROM_ADDR_COUNT+1, hi);
  eew_write(EEPROM_ADDR_CSUM, csum(EEPROM_MAGIC_VAL,lo,hi));
}
static uint16_t eeprom_load_target(void){
  uint8_t m=eer_read(EEPROM_ADDR_MAGIC), lo=eer_read(EEPROM_ADDR_COUNT),
hi=eer_read(EEPROM_ADDR_COUNT+1), cs=eer_read(EEPROM_ADDR_CSUM);
  if(m==EEPROM_MAGIC_VAL && cs==csum(m,lo,hi)){
    uint16_t v=(uint16_t)lo|((uint16_t)hi<<8);
    if(v>=1 && v<=1000) return v;
  }
  return 15;
}
static void eeprom_reset_defaults(void){ eeprom_save_target(15); }

// ========================= SSD1306 (minimal text) ============================
#define SSD1306_ADDR 0x3C
static void ssd_cmd(uint8_t c){ if(!i2c_start((SSD1306_ADDR<<1)|0)) return; i2c_write(0x00);
i2c_write(c); i2c_stop(); }
static void ssd_data_begin(void){ i2c_start((SSD1306_ADDR<<1)|0); i2c_write(0x40); }
static void ssd_data_end(void){ i2c_stop(); }
static void ssd_set_page_col(uint8_t x,uint8_t page){
  ssd_cmd(0xB0|(page&7)); ssd_cmd(0x00|(x&0x0F)); ssd_cmd(0x10|((x>>4)&0x0F));
}
static void ssd_init(void){
  __delay_ms(20);
```

```
  ssd_cmd(0xAE); ssd_cmd(0xD5); ssd_cmd(0x80);
  ssd_cmd(0xA8); ssd_cmd(0x3F); ssd_cmd(0xD3); ssd_cmd(0x00);
  ssd_cmd(0x40); ssd_cmd(0x8D); ssd_cmd(0x14);
  ssd_cmd(0x20); ssd_cmd(0x00); // horizontal
  ssd_cmd(0xA1); ssd_cmd(0xC8);
  ssd_cmd(0xDA); ssd_cmd(0x12);
  ssd_cmd(0x81); ssd_cmd(0x7F);
  ssd_cmd(0xD9); ssd_cmd(0xF1);
  ssd_cmd(0xDB); ssd_cmd(0x40);
  ssd_cmd(0xA4); ssd_cmd(0xA6); ssd_cmd(0x2E);
  // Clear
  for(uint8_t p=0;p<8;p++){ ssd_set_page_col(0,p); ssd_data_begin(); for(uint8_t i=0;i<128;i++) i2c_write(0x00); ssd_data_end(); }
  ssd_cmd(0xAF);
}
static void ssd_clear(void){
  for(uint8_t p=0;p<8;p++){ ssd_set_page_col(0,p); ssd_data_begin(); for(uint8_t i=0;i<128;i++) i2c_write(0x00); ssd_data_end(); }
}
// Compact 6x8 uppercase+digits font for essential UI (32..95 only; unknown -> '?')
static const uint8_t FONT6x8[64][6] = {
  // 32 ' ' .. 95 '_'
  {0,0,0,0,0,0},        // 32 ' '
  {0,0,0,0,0,0},        // 33 '!' (unused)
  {0,0,0,0,0,0},        // 34 '"'
  {0,0,0,0,0,0},        // 35 '#'
  {0,0,0,0,0,0},        // 36 '$'
  {0,0,0,0,0,0},        // 37 '%'
  {0,0,0,0,0,0},        // 38 '&'
```

```
{0,0,0,0,0,0},          // 39 '''
{0,0,0,0,0,0},          // 40 '('
{0,0,0,0,0,0},          // 41 ')'
{0,0,0,0,0,0},          // 42 '*'
{0,0,0,0,0,0},          // 43 '+'
{0,0,0,0,0,0},          // 44 ','
{0,0,0,0,0,0},          // 45 '-'
{0,0,0,0,0,0},          // 46 '.'
{0x10,0x10,0x10,0x10,0x10,0},// 47 '/' (simple vertical)
{0x3E,0x51,0x49,0x45,0x3E,0},// 48 '0'
{0x00,0x42,0x7F,0x40,0x00,0},// 49 '1'
{0x42,0x61,0x51,0x49,0x46,0},// 50 '2'
{0x21,0x41,0x45,0x4B,0x31,0},// 51 '3'
{0x18,0x14,0x12,0x7F,0x10,0},// 52 '4'
{0x27,0x45,0x45,0x45,0x39,0},// 53 '5'
{0x3C,0x4A,0x49,0x49,0x30,0},// 54 '6'
{0x01,0x71,0x09,0x05,0x03,0},// 55 '7'
{0x36,0x49,0x49,0x49,0x36,0},// 56 '8'
{0x06,0x49,0x49,0x29,0x1E,0},// 57 '9'
{0,0,0,0,0,0},          // 58 ':'
{0,0,0,0,0,0},          // 59 ';'
{0,0,0,0,0,0},          // 60 '<'
{0x08,0x08,0x08,0x08,0x08,0},// 61 '='
{0,0,0,0,0,0},          // 62 '>'
{0,0,0,0,0,0},          // 63 '?'
{0,0,0,0,0,0},          // 64 '@'
{0x7E,0x11,0x11,0x11,0x7E,0},// 65 'A'
{0x7F,0x49,0x49,0x49,0x36,0},// 66 'B'
{0x3E,0x41,0x41,0x41,0x22,0},// 67 'C'
```

```
    {0x7F,0x41,0x41,0x22,0x1C,0}, // 68 'D'
    {0x7F,0x49,0x49,0x49,0x41,0}, // 69 'E'
    {0x7F,0x09,0x09,0x09,0x01,0}, // 70 'F'
    {0x3E,0x41,0x49,0x49,0x7A,0}, // 71 'G'
    {0x7F,0x08,0x08,0x08,0x7F,0}, // 72 'H'
    {0x41,0x41,0x7F,0x41,0x41,0}, // 73 'I'
    {0x20,0x40,0x41,0x3F,0x01,0}, // 74 'J'
    {0x7F,0x08,0x14,0x22,0x41,0}, // 75 'K'
    {0x7F,0x40,0x40,0x40,0x40,0}, // 76 'L'
    {0x7F,0x02,0x0C,0x02,0x7F,0}, // 77 'M'
    {0x7F,0x04,0x08,0x10,0x7F,0}, // 78 'N'
    {0x3E,0x41,0x41,0x41,0x3E,0}, // 79 'O'
    {0x7F,0x09,0x09,0x09,0x06,0}, // 80 'P'
    {0x3E,0x41,0x51,0x21,0x5E,0}, // 81 'Q'
    {0x7F,0x09,0x19,0x29,0x46,0}, // 82 'R'
    {0x46,0x49,0x49,0x49,0x31,0}, // 83 'S'
    {0x01,0x01,0x7F,0x01,0x01,0}, // 84 'T'
    {0x3F,0x40,0x40,0x40,0x3F,0}, // 85 'U'
    {0x1F,0x20,0x40,0x20,0x1F,0}, // 86 'V'
    {0x7F,0x20,0x18,0x20,0x7F,0}, // 87 'W'
    {0x63,0x14,0x08,0x14,0x63,0}, // 88 'X'
    {0x07,0x08,0x70,0x08,0x07,0}, // 89 'Y'
    {0x61,0x51,0x49,0x45,0x43,0}, // 90 'Z'
    {0,0,0,0,0,0},              // 91 '['
    {0,0,0,0,0,0},              // 92 '\'
    {0,0,0,0,0,0},              // 93 ']'
    {0,0,0,0,0,0},              // 94 '^'
    {0x40,0x40,0x40,0x40,0x40,0}, // 95 '_'
};
```

```
static void ssd_draw_char(uint8_t x,uint8_t page,char c){
   uint8_t idx;
   if(c>=32 && c<=95) idx=(uint8_t)(c-32); else idx=(uint8_t)('?' - 32);
   ssd_set_page_col(x,page);
   ssd_data_begin();
   for(uint8_t i=0;i<5;i++) i2c_write(FONT6x8[idx][i]);
   i2c_write(0x00);
   ssd_data_end();
}
static void ssd_print(uint8_t x,uint8_t page,const char* s){
   uint8_t cx=x, pg=page;
   while(*s){
      if(*s=='\n'){ pg++; cx=x; } else { ssd_draw_char(cx,pg,(char)toupper((unsigned char)*s)); cx+=6;
if(cx>122){ cx=0; pg++; } }
      s++;
   }
}

// ========================= Command parser ==================================
typedef struct {
   uint16_t targetN;
   uint16_t liveCount;
   bool    modeAuto;
   bool    openRelayOn;
   bool    closeRelayOn;
   bool    openStatus;
   bool    closeStatus;
} sys_status_t;
```

```
static uint16_t g_targetN = 15;
static char     cmd_line[32];
static uint8_t  cmd_idx=0;

static void cmd_init(void){ g_targetN = eeprom_load_target(); }

static void cmd_set_target(uint16_t n){
   if(n<1) n=1; if(n>1000) n=1000;
   if(g_targetN!=n){ g_targetN=n; eeprom_save_target(g_targetN); }
}
static bool cmd_poll(sys_status_t*st){
   bool did=false;
   while(uart_kbhit()){
      char c=uart_getc();
      if(c=='\r'||c=='\n'){
         cmd_line[cmd_idx]=0; cmd_idx=0;
         if(cmd_line[0]){
            // uppercase
            for(char*p=cmd_line;*p;p++) *p=(char)toupper((unsigned char)*p);
            if(strncmp(cmd_line,"PULSE",5)==0){
               int n=atoi(&cmd_line[5]);
               if(n>=1 && n<=1000){ cmd_set_target((uint16_t)n); uart_puts("OK PULSE SET\r\n"); }
               else uart_puts("ERROR PULSE RANGE 1..1000\r\n");
            } else if(strcmp(cmd_line,"STS")==0){
               char b[96];
               snprintf(b,sizeof(b),
                  "MODE=%s RELAY_OPEN=%d RELAY_CLOSE=%d STS_OPEN=%d STS_CLOSE=%d
TARGET=%u LIVE=%u\r\n",
                  st->modeAuto?"AUTO":"MANUAL",
```

```
                st->openRelayOn?1:0, st->closeRelayOn?1:0,
                st->openStatus?1:0, st->closeStatus?1:0,
                st->targetN, st->liveCount);
            uart_puts(b);
          } else if(strcmp(cmd_line,"RESETMEM")==0){
            eeprom_reset_defaults(); g_targetN=eeprom_load_target(); uart_puts("OK RESET\r\n");
          } else {
            uart_puts("ERROR UNKNOWN CMD\r\n");
          }
          did=true;
        }
      } else {
        if(cmd_idx<sizeof(cmd_line)-1) cmd_line[cmd_idx++]=c;
      }
    }
    return did;
}

// ========================= System / ISR ====================================
typedef enum { DIR_IDLE=0, DIR_OPEN=1, DIR_CLOSE=2 } direction_t;
static volatile uint16_t liveCount=0;
static volatile uint32_t msTicks=0;
static volatile uint16_t lastEdgeTime=0;
static volatile bool moving=false;
static volatile direction_t dir=DIR_IDLE;
#define LS_DEBOUNCE_MS 8

void __interrupt() isr(void){
    if(PIR1bits.TMR2IF){ PIR1bits.TMR2IF=0; msTicks++; if((msTicks%500U)==0U) LAT_LED_HEART^=1; }
```

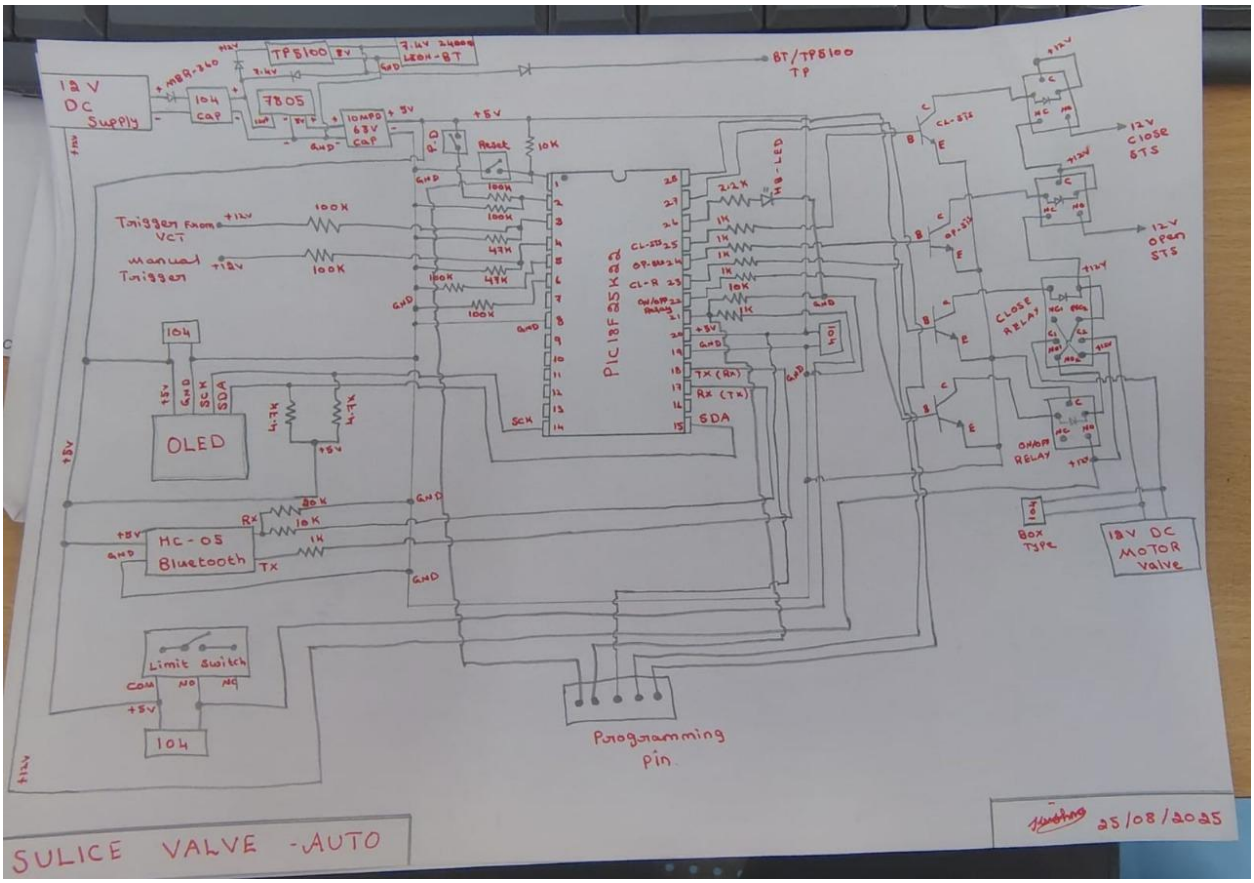
```
    if(INTCONbits.INT0IF){
        INTCONbits.INT0IF=0;
        uint16_t now=(uint16_t)(msTicks&0xFFFF), dt=(uint16_t)(now-lastEdgeTime);
        if(dt>=LS_DEBOUNCE_MS){
            lastEdgeTime=now;
            if(moving){
                if(dir==DIR_OPEN){ if(liveCount<1000) liveCount++; }
                else if(dir==DIR_CLOSE){ if(liveCount>0) liveCount--; }
            }
        }
    }
}

static void clock_init(void){
    OSCCON=0b01110010; while(!OSCCONbits.HFIOFS); // 16MHz INTOSC
}
static void gpio_init(void){
    ANSELA=0; ANSELB=0; ANSELC=0;
    TRIS_POWER_DETECT=1;
    TRIS_TRIGGER_AUTO=1;
    TRIS_MANUAL_TRIG =1;
    TRIS_OPEN_FB     =1;
    TRIS_CLOSE_FB    =1;
    TRIS_LIMIT_INT0  =1;

    TRIS_OPEN_RELAY  =0; LAT_OPEN_RELAY =0;
    TRIS_CLOSE_RELAY =0; LAT_CLOSE_RELAY=0;
    TRIS_OPEN_STATUS =0; LAT_OPEN_STATUS=0;
    TRIS_CLOSE_STATUS=0; LAT_CLOSE_STATUS=0;
```

```
    TRIS_LED_HEART   =0; LAT_LED_HEART  =0;

    // INT0 rising edge
    INTCON2bits.INTEDG0=1; INTCONbits.INT0IF=0; INTCONbits.INT0IE=1;
}
static void timer2_init_1ms(void){
    // Fosc/4=4MHz; prescale 1:16 => 250kHz; PR2=249 => 1kHz
    T2CON=0; T2CONbits.T2CKPS=0b11; T2CONbits.TMR2ON=1; // prescale 1:16
    PR2=249; PIR1bits.TMR2IF=0; PIE1bits.TMR2IE=1;
}
static void irq_enable(void){ RCONbits.IPEN=0; INTCONbits.PEIE=1; INTCONbits.GIE=1; }

static void stop_and_set_status(bool open_state){
    moving=false; dir=DIR_IDLE;
    LAT_OPEN_RELAY=0; LAT_CLOSE_RELAY=0;
    LAT_OPEN_STATUS = open_state?1:0;
    LAT_CLOSE_STATUS= open_state?0:1;
}
static void set_dir(direction_t d, uint16_t startCount){
    dir=d; moving=(d!=DIR_IDLE);
    if(d==DIR_OPEN){
        LAT_CLOSE_RELAY=0; __delay_ms(5);
        liveCount=startCount; LAT_OPEN_RELAY=1;
    }else if(d==DIR_CLOSE){
        LAT_OPEN_RELAY=0; __delay_ms(5);
        liveCount=startCount; LAT_CLOSE_RELAY=1;
    }else{
        LAT_OPEN_RELAY=0; LAT_CLOSE_RELAY=0;
    }
```

```
}

// ========================= Main ============================================
int main(void){
   clock_init(); gpio_init(); timer2_init_1ms(); i2c_init(100000); uart_init(9600); irq_enable();
   ssd_init(); ssd_clear();

   // Boot splash
   ssd_print(0,1,"REALTECH SYSTEMS");
   __delay_ms(800);
   ssd_clear();

   cmd_init(); // loads g_targetN from EEPROM
   stop_and_set_status(false); // initial: CLOSE status lit

   uint32_t lastUi=0;
   const uint32_t uiPeriodMs=100;

   while(1){
      bool modeAuto = (PIN_POWER_DETECT!=0);

      // Prepare status for STS reply
      sys_status_t st = {
         .targetN=g_targetN, .liveCount=liveCount, .modeAuto=modeAuto,
         .openRelayOn=(LAT_OPEN_RELAY!=0), .closeRelayOn=(LAT_CLOSE_RELAY!=0),
         .openStatus=(LAT_OPEN_STATUS!=0), .closeStatus=(LAT_CLOSE_STATUS!=0)
      };
      if(cmd_poll(&st)){ /* target may have changed */ }
```

```
if(modeAuto){
  // ========================= AUTO MODE =========================
  bool trig = (PIN_TRIGGER_AUTO!=0);
  if(trig){
    if(dir!=DIR_OPEN)  set_dir(DIR_OPEN, 0);
    if(liveCount>=g_targetN) stop_and_set_status(true);
  }else{
    if(dir!=DIR_CLOSE) set_dir(DIR_CLOSE, g_targetN);
    if(liveCount==0)   stop_and_set_status(false);
  }
}else{
  // ========================= MANUAL MODE (corrected) ==========
  // NO default relay action; NO EEPROM writes here.
  bool mtrig = (PIN_MANUAL_TRIG!=0); // If you want "always react" remove this gate.

  if(mtrig){
    if(PIN_OPEN_FB){    // If open feedback is HIGH → CLOSE until N
      if(dir!=DIR_CLOSE) set_dir(DIR_CLOSE, g_targetN);
    } else if(PIN_CLOSE_FB){ // If close feedback is HIGH → OPEN until N
      if(dir!=DIR_OPEN)  set_dir(DIR_OPEN, 0);
    }
  }

  // Stop conditions (purely by count)
  if(dir==DIR_CLOSE && liveCount==0){
    stop_and_set_status(false);
  } else if(dir==DIR_OPEN && liveCount>=g_targetN){
    stop_and_set_status(true);
  }
```

```
    }

    // Safety: never allow both relays on
    if(LAT_OPEN_RELAY && LAT_CLOSE_RELAY){
      LAT_OPEN_RELAY=0; LAT_CLOSE_RELAY=0; moving=false; dir=DIR_IDLE;
    }

    // OLED UI (minimal)
    if((msTicks-lastUi)>=uiPeriodMs){
      lastUi=msTicks;
      // Line0: MODE
      ssd_print(0,0, modeAuto ? "MODE: AUTO  " : "MODE: MANUAL");
      // Line1: STATUS
      if(LAT_OPEN_STATUS)     ssd_print(0,1,"STS-OPEN ");
      else if(LAT_CLOSE_STATUS)ssd_print(0,1,"STS-CLOSE");
      else                    ssd_print(0,1,"STS-???? ");
      // Line2: PULSE
      char b[24]; snprintf(b,sizeof(b),"PULSE %3u/%3u", (unsigned)liveCount, (unsigned)g_targetN);
      ssd_print(0,2,b);
      // Line3: ACTIVITY
      if(dir==DIR_OPEN)       ssd_print(0,3,"OPEN ON ");
      else if(dir==DIR_CLOSE)  ssd_print(0,3,"CLOSE ON");
      else                    ssd_print(0,3,"IDLE    ");
    }
  }
  // not reached
  // return 0;
}
```


12 V DC Supply
104 cap
7805
TP5100
7.4V 2400mA LEION-BT
BT/TP5100 TP
10MPS 63V cap
Reset
10K
Trigger from VCT
Manual Trigger
100K
47K
PIC18F25K22
OLED
SCK
SDA
4.7K
HC-05 Bluetooth
Rx
Tx
20K
10K
1K
Limit switch
COM
NO
NC
104
Programming pin
2.2K
HB-LED
CL-ST
OP-ST
CL-R
OP-R
ON/OFF
Analog
TX (Rx)
Rx (Tx)
SDA
401
12V CLOSE STS
12-V OPEN STS
CLOSE RELAY
ON/OFF RELAY
Box Type
12V DC MOTOR Valve
SULICE VALVE -AUTO
25/08/2025